
🧪 Kubernetes Hands-On Lab

Taints, Tolerations & Scheduling (Deep Practice)

📌 Lab Overview

In this lab, you will **practically explore how Kubernetes taints and tolerations work**, how the scheduler enforces them, and how certain configurations (like nodeName) bypass scheduling rules.

This lab intentionally creates **failures first**, then fixes them — the best way to truly understand scheduling behavior.

🎯 Learning Objectives

By the end of this lab, you will be able to:

- Add and remove **taints** on nodes
- Observe pod **scheduling failures**
- Configure **tolerations** correctly
- Understand **NoSchedule vs NoExecute**
- See how nodeName bypasses the scheduler
- Debug scheduling issues using kubectl describe
- Clean up taints safely

📋 Prerequisites

- A running Kubernetes cluster (kOps / EKS / AKS / GKE / Minikube)
- kubectl configured

- At least **one worker node**
- Basic understanding of Pods and Nodes

Verify cluster:

kubectl get nodes

🧠 Lab Scenario

You want to **reserve a node for special workloads** (GPU, critical apps, team-specific pods).

You will:

1. Taint a node 🚫
2. Watch pods fail ❌
3. Add tolerations 🎟️
4. Observe successful scheduling ✅
5. Test eviction behavior 💥
6. Learn dangerous bypasses ⚠️

🟢 Step 1: Identify a Node

List nodes:

kubectl get nodes

Choose one node (example):

i-0b903b7988f03793f

> 📌 We'll refer to this as **NODE-A** throughout the lab.

🚷 Step 2: Taint the Node (Create the Restriction)

Add a NoSchedule taint

kubectl taint nodes NODE-A role=special:NoSchedule

Example:

kubectl taint nodes i-0b903b7988f03793f role=special:NoSchedule

Verify taint

kubectl describe node NODE-A | grep -i taints

Expected output:

role=special:NoSchedule

🧠 **Meaning:**
Only pods with the correct toleration can be scheduled here.

❌ Step 3: Pod WITHOUT Toleration (Expected Failure)

Create a pod

kubectl run no-toleration-pod --image=nginx

Check pod status

kubectl get pods

Expected:

no-toleration-pod   Pending

Diagnose failure

kubectl describe pod no-toleration-pod

Expected message:

node(s) had taint {role: special}, that the pod didn't tolerate

✅ **This confirms the taint is working.**

🎟️ **Step 4: Pod WITH Toleration (Expected Success)**

Create pod YAML

toleration-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: toleration-pod
spec:
  tolerations:
  - key: "role"
    operator: "Equal"
    value: "special"
    effect: "NoSchedule"
  containers:
  - name: nginx
    image: nginx
```

Apply pod

kubectl apply -f toleration-pod.yaml

Verify scheduling

kubectl get pod toleration-pod -o wide

Expected:

STATUS: Running
NODE: NODE-A

🎉 **VIP access granted**

🚁 **Step 5: nodeName (Scheduler Bypass)**

⚠️ This step demonstrates **dangerous but important behavior**.

Create pod with nodeName

nodename-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nodename-pod
spec:
  nodeName: NODE-A
  containers:
  - name: nginx
    image: nginx
```

Apply pod

kubectl apply -f nodename-pod.yaml

Check pod

kubectl get pod nodename-pod -o wide

Expected:

STATUS: Running

🧠 **Key Lesson:**
nodeName **skips the scheduler**, so **taints are NOT enforced**.

⏳ **Step 6: NoExecute Taint (Eviction Test)**

Create a normal pod

kubectl run eviction-test --image=nginx

Ensure it is running:

kubectl get pod eviction-test -o wide

Apply NoExecute taint

kubectl taint nodes NODE-A env=prod:NoExecute

Watch pods

kubectl get pods -w

Expected:

eviction-test   Terminating

💥 **Pod is evicted immediately**

⏱️ **Step 7: NoExecute with Grace Period**

Create pod with tolerationSeconds

grace-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: grace-pod
spec:
  tolerations:
  - key: "env"
    operator: "Equal"
    value: "prod"
    effect: "NoExecute"
    tolerationSeconds: 30
  containers:
  - name: nginx
    image: nginx
```

Apply pod

kubectl apply -f grace-pod.yaml

⏳ Pod runs for **30 seconds**, then is evicted.

🧹 Step 8: Cleanup (Mandatory)

Delete pods:

kubectl delete pod --all

Remove taints:

kubectl taint nodes NODE-A role=special:NoSchedule-
kubectl taint nodes NODE-A env:NoExecute-

Verify:

kubectl describe node NODE-A | grep -i taints

Expected:

Taints: <none>

🧠 Key Takeaways

Scenario	Scheduler Used	Taints Enforced	Result
Normal pod	✅	✅	Blocked
Pod with toleration	✅	✅	Allowed

Pod with nodeName	❌	❌	Always runs
NoExecute taint	✅	✅	Eviction

🎯 Practice Challenges (Self-Test)

1. Change NoSchedule → PreferNoSchedule
2. Use **wrong toleration value** and observe failure
3. Combine **nodeSelector + toleration**
4. Taint **all nodes**, then recover cluster
5. Explain why **DaemonSets ignore NoSchedule**

✅ Final Notes

- **Never use nodeName in production** (except rare debugging cases)
- Taints **do not pull pods**, they only repel
- Tolerations **allow**, but do not force scheduling
- Always debug with:

kubectl describe pod <pod-name>